Blasted Studio

LIVE on Vultr

VULTR · AGENT RUSH HACKATHON — CHALLENGE 1

Sandboxed Agent Studio

AI agents that do **real work** — plan, write, and execute code inside isolated, disposable sandboxes on Vultr. Verifiable results. Blast radius: zero.

Data Analyst | Invoice Reconciler | Incident Investigator | multi-file upload

http://45.76.2.228 github.com/ASAYMAN69/blasted Docker · FastAPI · React · Vultr Serverless Inference

THE PROBLEM

AI can describe work. Can it be trusted to do it?

Real business value requires executing AI-generated code — analyzing data, reconciling invoices, investigating incidents. But running untrusted model output is dangerous. There is no isolation, no verification, and no proof.

Outputs

Models often *describe* results. Descriptions are not evidence.

Escape

AI code on the host can read secrets, hit the network, or crash the app.

Trust

No verification means no confidence that the result is real or correct.

Proof

Analysts need artifacts — charts, diffs, JSON verdicts — they can audit.

The business case: AI code-execution platforms are a real, growing market. The differentiator is **safe, verifiable execution with proof** — run untrusted AI code without risk.

THE SOLUTION

A centralized control layer that plans, executes, and verifies

Users submit a task in plain English — with data files dropped in, up to 20 at once. The agent plans, writes Python, and runs it inside an isolated Docker sandbox on Vultr — then returns the actual outputs: charts, verdicts, diffs, and downloadable artifacts.

01

Real work

Code executes in a real sandbox. Output is returned — not a description of what might have happened.

02

Verifiable

Each step is judged by deterministic rules: exit code, artifacts, structured JSON. Not the model's opinion.

03

Contained

Non-root, read-only filesystem, no network, resource limits, and **container destroyed after every task**.

Three agents, one engine. A shared agentic loop powers Data Analyst, Invoice Reconciler, and Incident Investigator — pluggable mode registry.

Self-healing. On failure, the previous error is fed back; the agent fixes and re-runs — bounded retries, never an infinite loop.

Production-style. React SPA, FastAPI, SSE live progress, artifact viewer, run history, CI/CD auto-deploy.

ARCHITECTURE

Everything runs on one Vultr VM

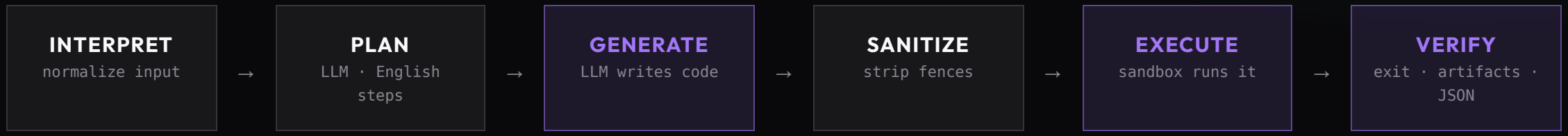
Vultr Cloud Compute hosts the web app, the LLM client, and the Docker sandbox runtime. The engine lives *outside* the sandbox; AI-generated code never touches the app process.



THE BRAIN

An agent is a loop — not magic

Each task walks the same state machine. Plan and generate are separate LLM calls; verification is deterministic and never delegated to the model.



Failure → bounded retry

If a step fails, the error is fed into the prompt and the agent regenerates — default 3 attempts, then an honest explanation instead of an infinite loop.

Explain + finalize

The LLM summarizes the real output in one terse sentence. Artifacts are copied out of the sandbox, the run card is persisted, and the SSE stream closes.

BLAST RADIUS ZERO — THE DIFFERENTIATOR

Containment is the product

Eight guarantees, enforced on every run, proven by 12 containment negative tests. The sandbox is a disposable container that never outlives its task.

GUARANTEE	IMPLEMENTATION	PROOF
Process isolation	Separate Docker container — never the app process	`host FS not mounted`
Non-root	Runs as `runner` (uid 1000)	`os.getuid() → 1000`
Read-only rootfs	`--read-only`; only `/workspace` + tmpfs writable	`/etc write → EROFS`
No network	`--network none` — zero egress	`urlopen → NET_BLOCKED`
Resource limits	512 MB memory · 0.5 CPU · 60s timeout	`OOM exit 137 · timeout exit 124`
Secret hygiene	No API keys ever enter the sandbox; LLM calls happen outside	`env vars absent`
Lifecycle	Container force-removed on success, error, timeout, OOM	`0 leftover containers`
Artifacts	Copied to `data/artifacts/` before destroy	`GET /api/artifacts/… 200`

CAPABILITIES — THREE AGENTS

Three multi-step agents, one engine

Each agent chains real executions — every step runs in its own sandbox, its output feeds the next step, and every result is persisted as a downloadable artifact.

Data Analyst

data-analyst

`input · CSV (multi-file, up to 20) + question`

`profile → clean → analyze → chart → report`

Question-driven — answers a specific question with one chart. Multi-file CSV profiling, cleaning, statistics, and a report.

- `chart.png`
- `result.json`
- `report`

Invoice Reconciler

invoice-reconciler

`input · invoices (---DOCUMENT--- wrapped) + PO CSV`

`extract → parse → reconcile → report → summarize`

Flags discrepancies — e.g. a $750 PO vs a $800 invoice — and produces a reconciliation summary.

- `reconcile.json`
- `report`
- `summary`

Incident Investigator

incident-investigator

`input · metrics CSV + events CSV (multi-file)`

`profile → detect anomalies → chart → correlate → hypothesize → verify fix → report`

Time-series chart highlights anomaly spikes; correlates events, hypothesizes root cause, verifies the fix in a sandbox.

- `chart.png`
- `correlate.json`
- `hypothesize.json`
- `verify_fix.json`
- `report.txt`

drag & drop

Up to 20 files on every file field — with per-file size, name, and content preview.

HOW VULTR POWERS THE SOLUTION

Vultr is the central control plane

Three Vultr services carry the whole system — from orchestration to inference to execution containment.

01

Cloud Compute VM

2 vCPU / 4 GB Ubuntu VM runs FastAPI, nginx, Docker, and systemd. The web app, engine, and sandbox runtime all live here — the single control point.

02

Serverless Inference

OpenAI-compatible LLM endpoint (`api.vultrinference.com/v1`, model `mimo-v2.6-flash`) powers plan generation, code generation, and result summarization.

03

Docker sandboxes

Per-task disposable containers on the VM: non-root, read-only rootfs, no network, resource limits, destroyed after every task. Firewall open to 22 / 80 / 443 only.

Live deployment — `http://45.76.2.228`, deployed 2026-11-03, CI/CD auto-deploy on merge.

Provider abstraction — LLM client is interface-driven; Vultr is primary, with Ollama / OpenRouter fallback.

Zero secrets in sandbox — credentials stay in `.env` on the host, never in the container.

ENGINEERING RIGOR

Proof, not claims

The containment story is backed by tests that actually run Docker and attempt to break the sandbox.

52	12	3	20
BACKEND TESTS PASSING	CONTAINMENT NEGATIVE TESTS	MULTI-STEP AGENTS	FILES PER UPLOAD

Containment negative tests

- Host filesystem is not mounted
- Container hostname isolated from host
- No host env / secrets leak into the sandbox
- Memory limit kills runaway allocation (exit 137)
- Timeout kills runaway loops (exit 124)
- Container destroyed after success / error / timeout / OOM

Verified end-to-end

- POST task → SSE loop → passed on attempt 1 (~5 s)
- Artifacts fetchable via API (chart.png → 200) — including text-only step outputs persisted as .json / .txt
- Multi-file drag & drop verified (up to 20 files per field)
- Empty-artifact state explains that text/JSON agents produce no binary files — never a misleading "no artifacts yet"
- CSV input-mapping bug caught and fixed by the suite

JUDGING ALIGNMENT

Trust through containment and evidence

Real execution · verifiable results · disposable sandboxes on Vultr. An AI agent that does real work you can safely run.

Technology	Originality	Business value	Presentation
Real LLM + real Docker execution + modular pipeline mode registry.	Containment proof, multi-step artifact chains, structured verdicts.	Safe AI code execution — untrusted model output without risk.	Live demo URL, this deck, demo video with a containment moment.

http://45.76.2.228 github.com/ASAYMAN69/blasted

VULTR AGENT RUSH HACKATHON · CHALLENGE 1 — BLAST RADIUS ZERO